

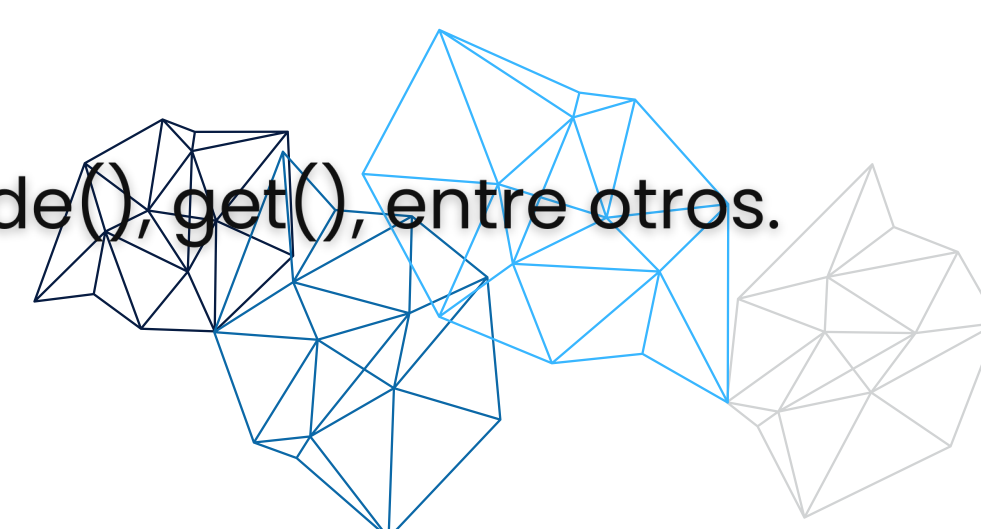
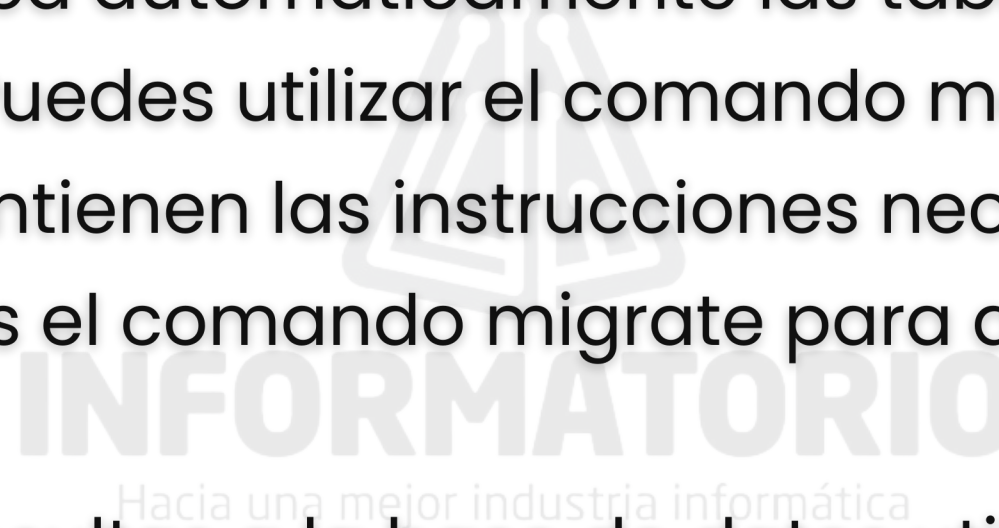
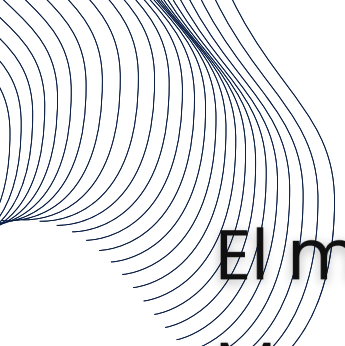
Conectar con BD

El manejo de la base de datos se realiza principalmente a través del ORM (Object-Relational Mapping) incorporado. El ORM de Django proporciona una abstracción de alto nivel para interactuar con la base de datos, lo que facilita la manipulación de los datos y la ejecución de consultas sin tener que escribir código SQL directamente. A continuación, te mostraremos algunas de las operaciones comunes de manejo de la base de datos en Django:

- Creación de tablas: Django crea automáticamente las tablas en la base de datos a partir de la definición de tus modelos. Puedes utilizar el comando `makemigrations` para generar los archivos de migración que contienen las instrucciones necesarias para crear las tablas en la base de datos. Luego, ejecutas el comando `migrate` para aplicar esas migraciones y crear las tablas.
- Consultas: Puedes realizar consultas a la base de datos utilizando métodos y filtros proporcionados por el ORM de Django. Por ejemplo, para obtener todos los objetos de un modelo, puedes usar el método `all()`:

```
objetos = MiModelo.objects.all()
```

Puedes aplicar filtros y condiciones utilizando métodos como `filter()`, `exclude()`, `get()`, entre otros.



- Guardar y actualizar objetos: Puedes crear nuevos objetos y guardarlos en la base de datos utilizando el método **save()**, o actualizar un objeto existente modificando sus atributos y luego llamando a **save()**.

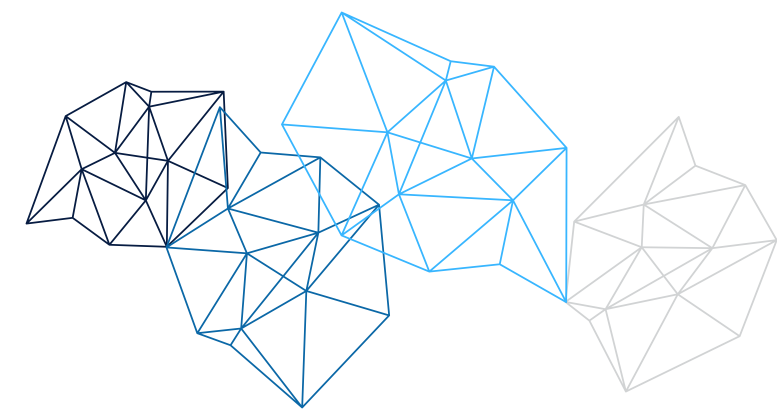
```
nuevo_objeto = MiModelo(campo1='valor1', campo2='valor2')
nuevo_objeto.save()
```

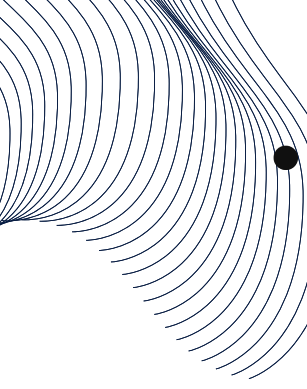
```
objeto_existente = MiModelo.objects.get(id=1)
objeto_existente.campo1 = 'nuevo_valor'
objeto_existente.save()
```

- Eliminación de objetos: Puedes eliminar objetos de la base de datos utilizando el método **delete()** en el objeto correspondiente.

```
objeto = MiModelo.objects.get(id=1)
objeto.delete()
```

INFORMATICA
Hacia una mejor industria informática

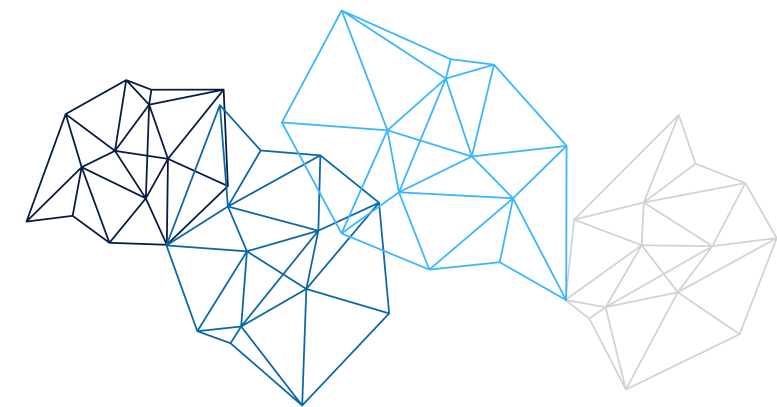


- 
- Relaciones: Puedes utilizar el ORM de Django para manejar relaciones entre modelos, como relaciones de clave externa (ForeignKey) y relaciones de muchos a muchos (ManyToManyField). Puedes acceder a los objetos relacionados a través de los atributos de los modelos y realizar operaciones de consulta y manipulación en ellos.

La conexión con la base de datos se establece a través de la configuración en el archivo **settings.py** de tu proyecto. A continuación te muestro cómo puedes configurar la conexión con la base de datos en Django:

- Abre el archivo settings.py en tu proyecto Django.
- Busca el diccionario DATABASES en el archivo. Si no existe, agrégalo al nivel de configuración principal.
- Dentro del diccionario DATABASES, puedes configurar una o más bases de datos según tus necesidades. El diccionario contendrá una clave para cada base de datos, y cada clave contendrá un subdiccionario con la configuración específica de la base de datos.
- Dentro del diccionario DATABASES, agrega una nueva clave para tu base de datos MySQL. Por ejemplo, podrías nombrarla 'default'

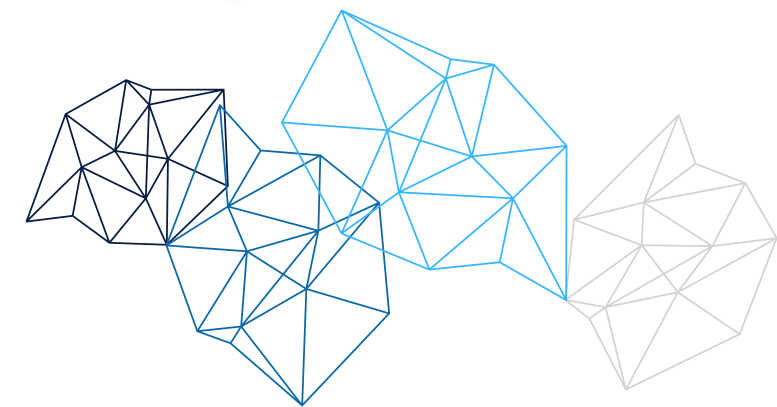
INFORMATARIO

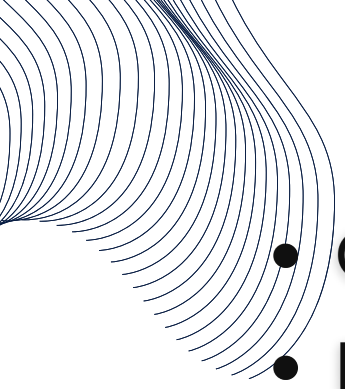


- Asegúrate de tener instalado el controlador de MySQL para Python. Puedes instalarlo utilizando el siguiente comando:
 - **pip install mysqlclient**
- Agrega los datos en el diccionario DATABASES

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.mysql',  
        'NAME': 'nombre_basedatos',  
        'USER': 'usuario_mysql',  
        'PASSWORD': 'contraseña_mysql',  
        'HOST': 'localhost',  
        'PORT': '3306',  
    }  
}
```

Asegúrate de reemplazar 'nombre_basedatos', 'usuario_mysql' y 'contraseña_mysql' con los valores correspondientes para tu base de datos MySQL. Si estás utilizando un host o un puerto diferente, ajusta los valores de 'HOST' y 'PORT' en consecuencia.



- 
- Guarda los cambios en **settings.py**.
 - Ejecuta el comando **makemigrations** para generar los archivos de migración:
  >**python manage.py makemigrations**
 - Ejecuta el comando **migrate** para aplicar las migraciones y crear las tablas en la base de datos:
  >**python manage.py migrate**

Con estos pasos, has configurado exitosamente una base de datos MySQL en tu proyecto Django. Puedes comenzar a utilizar el ORM de Django para interactuar con la base de datos y realizar operaciones de consulta, escritura y modificación de datos.

Hacia una mejor industria informática



Subsecretaría de
Empleo



Ministerio de
Producción, Industria y Empleo



CHACO
Gobierno de todos

